Agentic AI System Design

End-to-end architecture walkthroughs
Airbnb AI Trip Concierge & Apple Health AI Coach

Prepared by: Mahesh Zade

Completed: 13 October 2026

Guidance: Amit Harjani, Microsoft

02 · Executive summary

Two agentic systems, two different risk profiles

- Airbnb optimizes trusted commerce: discovery, live quotes, bookings, changes, policy answers and host messaging.
- Apple Health optimizes safe coaching: interpret derived health metrics, explain trends and route users toward appropriate care.
- Both use Clarify → Frame → Design → Trade-offs → Measure.
- Architecture is derived from blast radius, data sensitivity and reversibility—not from a tool catalogue.

03 · Project objectives

- Translate user journeys into explicit system requirements.
- Design Knowledge, Tools & Permissions, Architecture and Observability.
- Separate model judgment from deterministic policy and safety controls.
- Define measurable rollout gates, failure responses and rollback conditions.
- Demonstrate working Langflow-style flows using the supplied project artifacts.

04 · Design method

1. Clarify: Stakes, Inputs, Actions, Operations.
2. Frame: users, outcome, worst case and non-negotiables.
3. Design: four layers—Knowledge; Tools & Permissions; Architecture; Observability.
4. Trade-offs: state what is intentionally sacrificed and why.
5. Measure: instrument decision quality and connect signals to actions.

05 · Two cases at a glance

Airbnb Concierge

• 2M+ travelers; 220+ markets (assignment scenario).

• Money, bookings, cancellations, private trip data and untrusted host text.

Apple Health Coach

• Personal health metrics and coaching.

• Privacy boundary, deterministic anomaly logic, safety-gated language and consented actions.

06 · Case 1 — Airbnb: problem statement

Design an AI Trip Concierge that helps travelers discover and manage stays end-to-end: destination research, listing discovery, live price/availability, booking, cancellations, refund-policy answers, date changes and host messages.

Core design principle: cheap mistakes may tolerate uncertainty; money movement, cancellation and PII disclosure may not.

07 · Airbnb: clarify before design

Stakes: Can the agent commit money? Which errors cannot be undone?

Inputs: Is host-authored text in scope? It is untrusted.

Actions: Own trips only, or browse and book new inventory?

Operations: Which markets and legal rules apply from day one?

Each question must flip a downstream design decision.

08 · Airbnb: scope and assumptions

Primary user: traveler managing a trip.

Outcome: fast, correct, safe trip discovery and management.

Use cases: recommend, quote, policy answer, book, cancel/change, message host.

Worst cases: wrong charge/cancellation; leaked traveler or host PII; injected host content causing unsafe action.

Assumptions are proposals and should be calibrated with real operational data.

09 · Airbnb: success and non-negotiables

- Confirm and read back every spend or irreversible commitment.
- Cite policy answers or abstain when evidence is insufficient.
- Enforce identity, entitlement and region rules before retrieval/action.
- Treat listing descriptions, reviews and host messages as untrusted data.
- Re-check identity and authorization server-side at execution time.

10 · Airbnb: knowledge decision — RAG vs live APIs

Use RAG for stable, citable reference knowledge: cancellation policies, house rules and destination guides.

Use live APIs for perishable or personal state: price, availability, trip history and reservation state.

Rule: a price is a promise about now; never rely on a stale index for it.

Policy documents should be versioned and retrievable at clause level.

11 · Airbnb: answer contracts

Recommendation: explain why it fits and cite grounding; no commitment implied.

Price quote: listing ID, dates, total price, fees and cancellation tier from live source.

Policy answer: applicable clause, effective date and source link—or abstain.

Booking/change: summarize exact action, dates, amount and cancellation implications before confirmation.

12 · Airbnb: ingestion and freshness

- Cancellation/refund policy: authoritative source, versioned; re-ingest on policy update.
- Host house rules: listing-specific; refresh on host edit.
- Destination guides: batch refresh; days of staleness may be acceptable.
- Price and availability: live read-through, not indexed as truth.
- Traveler trip state: live per session, scoped to authenticated user.

13 · Airbnb: retrieval pipeline

Query → identity/region entitlement filter → lexical BM25 + dense retrieval → merge → rerank → augment with citations → answer or abstain.

Pre-filtering ensures unauthorized documents never enter model context. Keep indexes and metadata aligned with policy version, region, source authority and freshness.

14 · Airbnb: knowledge security

Threats: indirect prompt injection in listings, reviews and host messages; poisoned content entering an index; cross-user retrieval leakage.

Controls: treat retrieved text as data, isolate instructions from content, source allowlists, provenance and review, pre-retrieval entitlements, output checks, and adversarial evaluation.

15 · Airbnb: tool classification

READ: search listings, quote availability, read own reservation, retrieve policy.

WRITE: draft host message, save itinerary/preferences, create a reversible hold.

TRIGGER: book/pay, cancel, send host message, commit refund or reservation change.

Blast radius—not implementation convenience—determines permission tier.

16 · Airbnb: permission tiers

Auto-execute: low-risk reads and recommendations.

Confirmation + read-back: booking, payment, cancellation, date change and sending messages.

Human-approved: exceptional high-value, ambiguous, disputed or policy-sensitive cases.

Use step-up authentication where needed; server-side authorization is mandatory.

17 · Airbnb: constrained tools and identity

Model schema constrains fields, allowed actions and formats; server validates every argument independently.

Use scoped user tokens and explicit reservation/listing IDs.

Never trust model-supplied identity or price.

Idempotency keys prevent duplicate bookings; server rechecks current quote, permissions, inventory and policy before commit.

18 · Airbnb: Rule-of-Two / prompt injection

The agent can access private data, process untrusted text and act/spend—three risky capabilities together.

Break the dangerous chain: untrusted content cannot directly authorize external action.

Separate trusted instructions from host content; require trusted-UI confirmation for triggers; use least-privilege tools and allowlisted destinations.

19 · Airbnb: architecture diagram

ARCHITECTURE FLOW

Traveler App → Auth + Guardrails → Concierge Agent

Concierge Agent → Policy RAG / Live Travel APIs → Plan + Validate

Plan + Validate → Trusted Confirmation UI → Server Re-check → Booking / Messaging APIs → Audit + Metrics

User / Traveler App

↓

Authentication + input guardrails

↓

Single Concierge Agent (bounded loop)

■ Knowledge: Policy RAG + destination content ■ Live tools: search, quote, trip state

↓

Plan + validate proposed action

↓

Trusted confirmation UI for irreversible actions

↓

Server-side policy / identity / price checks

↓

Booking and messaging APIs

↓

Audit log + outcome monitoring

20 · Airbnb: end-to-end trip-change flow

Example: “Shift my Lisbon stay to next weekend and tell the host I’ll arrive late.”

1. Authenticate traveler and fetch their reservation.
2. Resolve “next weekend” and verify dates with the user if ambiguous.
3. Query live availability and price delta.
4. Retrieve cancellation/change policy and explain constraints.
5. Stage proposed modification and draft message.
6. Read back dates, price and message; obtain confirmation.
7. Server revalidates quote and authorization, commits once, then sends message.
8. Return confirmation IDs and audit event.

21 · Airbnb: single agent first

Start with one Concierge Agent and explicit tools.

Benefits: simpler context flow, fewer handoffs, easier tracing and lower orchestration overhead.

Add a specialized agent only after evidence of context overload, competing objectives or a genuinely distinct expertise boundary.

A multi-agent design is not automatically safer.

22 · Airbnb: memory and state

Short-term: current conversation and pending transaction state; expires with session or workflow.

Long-term: explicit traveler preferences only where consent and product policy allow.

Shared knowledge: versioned policies and destination guides, never personal trip state.

Per-user access control, retention limits and deletion pathways apply to all stored data.

23 · Airbnb: stopping and handoff

Stop when required facts are missing, quote expires, policy retrieval misses, authorization fails, tools disagree, or the user declines confirmation.

Show a clear status: completed, awaiting confirmation, blocked, or handed to support.

Never silently retry a financial trigger or leave the traveler uncertain whether a booking happened.

24 · Airbnb: observability

Measure decisions, not just uptime.

Safety: unauthorized charge, wrong cancellation, PII leakage, injection success.

Quality: quote accuracy, policy citation correctness, booking completion, task success and user corrections.

Operations: tool latency, timeout rate, stale quote rate, retry/idempotency conflicts and handoff rate.

25 · Airbnb: rollout and rollback

Proposed stages: offline eval → internal dogfood → read-only pilot → staged confirmed actions → broader market rollout.

Use market/feature flags and canary cohorts.

Any confirmed unauthorized charge or critical PII incident triggers immediate containment and review.

Numeric thresholds (including the assignment’s illustrative 0.1% line) are proposals to calibrate—not universal facts.

26 · Airbnb: trade-offs and what changes the design

Traded away: fully autonomous booking for lower friction; more latency for live quotes and confirmations; some answer coverage for citation reliability.

Change the design if: action error rates remain low under evidence, policy law changes, traffic exceeds single-agent context/latency envelope, or evaluations show a repeatable specialist boundary.

27 · Case 2 — Apple Health AI Coach

A privacy-preserving coach summarizes personal metrics, explains trends and helps users take appropriate next steps.

The LLM is not the detector of record and must not diagnose or prescribe.

Deterministic analysis identifies candidate anomalies; the coach explains them within strict safety boundaries.

28 · Apple Health: clarify and frame

Clarify: intended age groups and use context; which metrics and devices; what constitutes urgent escalation; what data may leave device; which external actions are permitted.

Frame: useful coaching with privacy by default and no false reassurance.

Worst case: missed serious signal, unsafe reassurance, sensitive data exposure or unauthorized action.

29 · Apple Health: architecture diagram

ON DEVICE: HealthKit/raw metrics → deterministic baseline and anomaly engines → derived context → coach SLM → safety gate → native consent UI.

OPTIONAL CLOUD: minimal derived/anonymized context → clinical guideline RAG → bounded response → on-device safety gate.

Consent UI → partner scheduling/test APIs only after explicit approval.

Raw health data stays on device by default.

30 · Apple Health: knowledge strategy

Use authoritative, versioned clinical/public-health guidance for general explanations; retain source and effective date.

Use on-device personal history for an individual’s baseline and trends.

Do not treat population guidance as a diagnosis of an individual.

If guideline evidence is unavailable or conflicting, abstain and direct the user to a qualified professional.

31 · Apple Health: deterministic analysis

ARCHITECTURE FLOW

ON DEVICE: HealthKit data → Deterministic baseline / anomaly engine → Coach SLM → Safety gate + consent UI

OPTIONAL CLOUD: Derived context → Clinical guideline RAG → bounded response

Consent UI → Partner API only after explicit approval

Baseline and anomaly detection should be deterministic and validated for the chosen metric/device.

LLM role: translate a result into clear, calibrated language and explain uncertainty.

Critical event routing must not depend solely on generative interpretation.

Account for sensor noise, missing measurements, device differences and baseline drift.

32 · Apple Health: safety policy

Never diagnose or prescribe.

Do not offer false reassurance when symptoms or signals may be urgent.

Use escalation language and appropriate emergency guidance when indicated.

Avoid inventing thresholds; thresholds require clinical validation and appropriate oversight.

The safety gate can block, rewrite or route an answer.

33 · Apple Health: permissions and tools

READ: summarize authorized metrics and trends.

WRITE: save user-approved goals or reminders.

TRIGGER: schedule an appointment or contact a partner.

External actions require clear consent through native UI; use minimum necessary data and explicit purpose limitation.

34 · Apple Health: privacy boundary

Raw HealthKit data remains on device by default.

Only minimum necessary derived context may be sent to a cloud service, if enabled and consented.

Separate identifiers from analytics; encrypt in transit and at rest; scope access; set retention and deletion rules.

Do not describe derived data as automatically anonymous without validating re-identification risk.

35 · Apple Health: architecture choices

Prefer on-device processing for privacy and low-latency coaching.

Use cloud reasoning only when it materially improves the response and consent allows it.

Fallback gracefully when offline: local summaries and safe, bounded guidance.

Do not let cloud unavailability suppress deterministic critical alert handling.

36 · Apple Health: user journey

1. User asks: “Why was my resting heart rate higher this week?”
2. Read authorized data locally and check data completeness.
3. Deterministic engine compares against personal baseline and quality rules.
4. Coach explains observed trend, uncertainty and non-diagnostic possibilities.
5. Safety gate checks claims and escalation wording.
6. If action is proposed, native consent UI requests approval.
7. Return explanation and clear next steps.

37 · Apple Health: observability

Safety: false negatives, false reassurance, unsafe recommendations, privacy boundary violations.

Quality: explanation helpfulness, groundedness, calibration and user comprehension.

Operations: on-device latency, cloud timeout, battery impact and data availability.

Monitor performance by device/metric cohort and investigate distribution shifts.

38 · Apple Health: incident response

A suspected critical miss triggers immediate clinical-safety review and feature containment.

Preserve minimal, privacy-respecting diagnostic evidence.

Review sensor quality, deterministic thresholds, routing and safety-gate behavior.

Do not assume a single universal numerical threshold; define thresholds with clinical governance and validation.

39 · Apple Health: trade-offs

ARCHITECTURE FLOW

User UI → Auth + Input Validation → Agent / Orchestrator

Agent ↔ Knowledge + Policy; Agent ↔ Constrained Tools

Safety / Confirmation + Validators → External systems; all key decisions → Audit + Telemetry

Privacy vs cloud capability: default local processing, optional derived context.

Sensitivity vs false alarms: calibrate thresholds with validated data and human clinical oversight.

Convenience vs consent: explicit approval for external actions.

Coverage vs safety: abstain when evidence is weak rather than manufacture certainty.

40 · Cross-case comparison

Airbnb: trust boundary centers on money, reservation state and untrusted third-party text.

Apple Health: trust boundary centers on health privacy, deterministic signal handling and non-diagnostic coaching.

Airbnb friction belongs at commit time; Apple Health safety checks belong at detection, response and action time.

Both require least privilege, provenance, explicit stop conditions and decision-level monitoring.

41 · Shared agentic reference architecture

User interface → authentication and input validation → orchestrator/agent → policy and knowledge services → constrained tools → deterministic validators → confirmation/safety gate → external systems → audit/telemetry.

Cross-cutting controls: secrets management, versioned prompts, least privilege, rate limits, retries with idempotency, evaluation and rollback.

42 · Langflow implementation: Airbnb flow

Supplied flow JSON contains Chat Input, Prompt Template, OpenAI model, Policy RAG Tool, Travel API Tool, Concierge Agent and Chat Output.

The prompt routes policy queries to policy RAG, trip/listing queries to the travel API and requires confirmation for high-stakes actions.

Current sample custom components return illustrative static values; replace with authenticated production APIs and real retrieval before claiming live booking capability.

43 · Langflow implementation: Apple Health flow

Supplied screenshot/flow shows Chat Input feeding a Tool Calling Agent with Health Vitals Tool and Clinical Scheduler Tool, using a selected language model and Chat Output.

The Health Vitals Tool summarizes metrics; the scheduler tool represents appointment scheduling.

Add explicit on-device privacy enforcement, deterministic anomaly engine, safety gate, native consent and audit controls around this prototype.

44 · Prototype versus production

Prototype proves orchestration shape—not production readiness.

Production additions: authentication/authorization, secure secret storage, validated tool schemas, network/API error handling, durable audit events, idempotency, consent UX, evaluation suite, monitoring and incident runbooks.

Never embed real API keys in exported flow JSON or prompts.

45 · Evaluation plan

Offline test sets: routine, ambiguous, adversarial, boundary and failure cases.

Airbnb: quote freshness, correct policy citation, authorization, confirmation adherence, duplicate booking prevention and injection resistance.

Apple Health: metric faithfulness, uncertainty calibration, safe escalation, non-diagnostic language, privacy leakage and tool consent.

Track regression results for every prompt, model, index and tool version.

46 · Threat model and controls

Threat: prompt injection → untrusted-content isolation and no direct action authority.

Threat: cross-user data leak → pre-retrieval entitlement and server authorization.

Threat: stale price → live quote + expiry + commit-time revalidation.

Threat: duplicate trigger → idempotency key and transaction state.

Threat: unsafe health advice → deterministic routing, output safety gate and clinician-reviewed policy.

47 · Reliability and failure handling

Timeout: bounded retry only for safe reads; do not blindly repeat triggers.

Tool disagreement: stop and surface the conflict.

Expired quote: refresh and ask for reconfirmation if price/action changes.

Model uncertainty: ask a clarifying question or abstain.

Cloud outage in health coach: preserve on-device safe functions and never disable critical deterministic routing.

48 · Security and governance

Least privilege for every tool and service identity.

Secrets managed outside flow files and source control.

PII/health data minimization, encryption, access logging and retention controls.

Versioned prompts, retrieval sources, schemas and safety policies.

Formal change approval and rollback ownership for high-risk changes.

49 · Program delivery plan

Phase 0: requirements, risk review and data inventory.

Phase 1: read-only prototype and synthetic fixtures.

Phase 2: evaluation harness and adversarial tests.

Phase 3: limited pilot with confirmation gates.

Phase 4: staged rollout with dashboards and on-call ownership.

Phase 5: post-launch review, drift checks and policy updates.

50 · RACI and operating model

Product: outcome, UX and rollout gates.

Engineering: APIs, architecture, reliability and release controls.

Security/privacy: threat model, access and retention review.

Clinical governance (Health): validate thresholds, escalation wording and safety cases.

Operations/support: incident response, user handoff and feedback loops.

Clear owner for every metric, alert and rollback switch.

51 · Metrics dashboard proposal

Airbnb: successful task completion; quote freshness; policy citation accuracy; unauthorized action count; booking/change error rate; handoff rate; p95 latency.

Apple Health: deterministic detection performance; safety-gate pass/fail; unsafe response rate; privacy incidents; explanation calibration; local/cloud latency.

Report denominators and confidence intervals; segment by market, device and task type.

52 · Risk register

R1 Wrong charge/change — confirmation, server revalidation, idempotency.

R2 PII exposure — entitlement pre-filter, data minimization, output checks.

R3 Prompt injection — untrusted content isolation, tool allowlists.

R4 Stale/incorrect policy — source versioning and citation contract.

R5 Missed health signal — deterministic validated detection and escalation review.

R6 Unsafe coaching — non-diagnostic policy, safety gate and clinical oversight.

53 · Pre-mortem: Airbnb

If launch failed, likely causes: stale price at commit, policy retrieval used wrong region/version, injected host content manipulated action, duplicate booking after retry, or unclear pending status.

Prevent with commit-time validation, region-scoped retrieval, trusted confirmation, idempotency and explicit state UX.

54 · Pre-mortem: Apple Health

If launch failed, likely causes: noisy sensor data interpreted as meaningful, cloud model gave false reassurance, health context leaked, urgent route depended on LLM output, or users mistook coaching for diagnosis.

Prevent with deterministic checks, safe abstention/escalation, on-device boundary, consent and clear product language.

55 · What would change my mind?

Airbnb: measured context overload, recurring cross-domain failures or a distinct re-accommodation objective may justify a specialist agent.

Apple Health: validated evidence that local processing cannot meet a specific requirement may justify limited cloud processing with consent.

In both: real user data, legal review, red-team results and measured error distributions should recalibrate proposed thresholds.

56 · Interview-ready 60-second summary

“I start from blast radius. For Airbnb, I separate stable policy knowledge from live price and reservation state, begin with one agent, and put every irreversible action behind trusted confirmation and server-side validation. For Apple Health, raw metrics stay on device by default, deterministic logic handles anomaly detection, and the LLM explains rather than diagnoses. In both systems, I instrument decision quality, define clear stop conditions, and tie safety signals to containment and rollback.”

57 · Project completion and acknowledgements

Project completed: 13 October 2026

Prepared by: Mahesh Zade

Guidance: Amit Harjani, Microsoft

Scope: Airbnb AI Trip Concierge + Apple Health AI Coach

Deliverables: architecture, permissions, knowledge strategy, observability, trade-offs, evaluation and rollout plan.

Thank you